

ProofGuard 1.0

Matthew Farah

July 13, 2026

Contents

1	System Version	2
2	Overview	2
3	Implementation	2
4	Expected Competition Performance	3
5	References	3

1 System Version

ProofGuard 1.0 (July 2026) is the first release of the system, and has been entered into ProoVer 2026. The E prover, built from current upstream sources, is bundled with the system as the backend ATP.

2 Overview

ProofGuard 1.0 is an automatic checker for first-order logic proof certificates submitted in the TSTP format [1]. A problem file and a proposed proof file are taken as input, and it is checked whether the proof establishes the claimed contradiction.

The system is designed for refutation proofs. A proof is accepted only when every step of the argument can be followed and it can be confirmed that **\$false** is correctly derived. When all required steps have been verified, the proof is reported as verified.

ProofGuard is intentionally conservative. When a proof step is unclear, unsupported, malformed, or cannot be checked within the available time, no guess is made. It is instead reported that the proof could not be verified. A proof is rejected as flawed only when a definite error can be identified, such as an incorrect negated conjecture, an invalid Skolemization step, or an inference which does not follow from its stated premises.

This design reflects the competition setting, in which the incorrect acceptance of a bad proof is heavily penalized. Caution is therefore prioritized, wherein only fully checked proofs are accepted, only demonstrably flawed proofs are rejected, and verification is otherwise reported as inconclusive.

Verdicts are reported using the required SZS statuses [2]. An accepted proof is reported as **VerifiedGood**; a rejected proof is reported as **VerifiedBad**, together with a one-line reason naming the flawed step; an inconclusive verification is reported as **Unknown**; and an exhausted time budget is reported as **Timeout**.

3 Implementation

- **Problem and proof parsing.** FOF problem files and TSTP proof files are parsed by a recursive-descent parser, and formulas are represented as immutable dataclass terms. Nested **inference(...)** chains are flattened to their leaf premises.
- **Input-step validation.** Steps imported from the problem file are checked against the referenced problem formula, modulo alpha-renaming and harmless syntactic normalizations such as literal order and equation orientation.
- **Internal rule checking.** Specified rules are verified internally. Negated-conjecture steps are checked against the negation of the original conjecture, which may be consumed by no other step. Skolemization steps are checked for fresh Skolem symbols, consistent annotations, and correct dependency on the universal variables in scope.
- **Conservative de-Skolemization fallback.** When Skolemization annotations are absent, de-Skolemization is attempted only when the transformation is unambiguous: each fresh Skolem term is replaced by a new existentially bound variable, after which the remaining theorem-status content is checked by the ATP. Whenever the transformation cannot be performed unambiguously, verification is reported as inconclusive.
- **ATP-backed consequence checking.** Unspecified inference steps marked with theorem status are checked through the generation of local proof obligations for the E prover [3]. The original conjecture is never independently re-proved.
- **Conservative failure policy.** A demonstrably incorrect step causes a failed verification to be reported. A malformed, unsupported, ambiguous, timed-out, or internally failing check causes verification to be reported as inconclusive; the system is never crashed by malformed input.

- **Resource management.** A global wall-clock budget is managed by the checker. Each ATP call is given a CPU limit derived from the remaining time, and the system is terminated conservatively before the external time limit is exceeded.
- **Packaging and reproducibility.** ProofGuard is implemented in pure Python 3 with no third-party Python libraries. The delivery package is built with Docker and is fully self-contained: the CPython runtime, the E binary, and the checker are all included, glibc 2.17 is targeted, and the system is launched as `./proover-check PROBLEM PROOF`.
- **Testing.** The system was tested on the published ProVer examples, on E-generated TSTP proofs over the TPTP library, on multi-system TSTP samples, and on mutation-fuzzed proof variants.
- **Availability.** The source is available [here](#).

4 Expected Competition Performance

All published example problems are classified correctly, and each evil proof is rejected for its intended flaw. Wrong answers are expected to be rare: across 58 TSTP proofs produced by five different systems, no correct proof was misclassified as flawed, and no mutation-fuzzed proof was ever accepted.

The strongest performance is expected on invalid proofs, for which structural checks and countersatisfiability results are obtained cheaply and definitively. On the published 8 MB stress sample, a flaw (a missing Skolem dependency at step 219 of 223) is located in roughly 12 seconds, without any ATP call on the large formulas.

Valid proofs written in the style of the published examples are expected to be verified within the time limit, while proofs relying on techniques outside the system’s coverage are reported as inconclusive rather than risked. Overall, it is expected that most lost points will be attributable to inconclusive verdicts rather than to penalties.

5 References

- [1] G. Sutcliffe: The TPTP Problem Library and Associated Infrastructure: From CNF to TH0, TPTP v6.4.0. *Journal of Automated Reasoning* 59(4), pp. 483–502, 2017.
- [2] G. Sutcliffe: The SZS Ontologies for Automated Reasoning Software. *Proc. LPAR Workshops, CEUR Workshop Proceedings* 418, pp. 38–49, 2008.
- [3] S. Schulz, S. Cruanes, P. Vukmirović: Faster, Higher, Stronger: E 2.3. *Proc. CADE-27, LNCS* 11716, pp. 495–507, Springer, 2019.

Disclosure: generative AI was used in part in the creation of this submission.